Ministry of Education
Imam Abdulrahman Bin Faisal University
Collage of Computer Sciences
and Information Technology
Department of Computer Sciences

جامعة الإمام عبد الرحمن بن فيصل
IMAM ABDULRAHMAN BIN FAISAL UNIVERSITY

Text Searching Using Naive and KMP Algorithms: A Comparative Study

Table of Contents

List of Tables

List of Figures

Abstract

The growth of the generation of digital documents and textual data on the web has increased the demand for efficient text searching. Searching algorithms have extensive applications in different fields like search engines, cyber security systems, text classification and information retrieval systems. Therefore, it is necessary to choose an efficient searching algorithm to improve the system performance and reduce the processing time.

This project is a comparison of two text searching algorithms, i.e. the Naive algorithm and the Knuth-Morris-Pratt (KMP) algorithm. The purpose of the study is to evaluate the algorithms in terms of execution time, efficiency and performance for various input cases and text sizes.

The experiments were performed on the 20 Newsgroups Dataset, which is composed of thousands of real-world text documents from different categories such as sports, politics, technology, and science. Different searching patterns and different input sizes were applied to study best case, average case and worst-case scenarios of both algorithms.

The Results obtained showed that the KMP algorithm gives better performance for large text datasets as it avoids unnecessary repeated comparisons during the searching process. On the other hand, Naive algorithm becomes slower with the increase of input size.

This project emphasizes the importance of efficient text searching algorithms and shows how the optimization of algorithms can greatly enhance text processing and information retrieval systems.

Section I: Theoretical Definition of the problem

1. Introduction

One of the most significant operations in Computer science and information retrieval systems is text searching. Searching algorithms are needed to accurately and quickly process large quantities of data, given the rapid proliferation in digital content and online text documents. Search engines, text classification systems, cybersecurity systems, and natural language processing are among the many applications that utilize text searching algorithms.

The purpose of this project is to compare two popular pattern searching algorithms namely Naive algorithm and Knuth-Morris-Pratt (KMP) algorithm. The two algorithms are both used to find a specific pattern or a specific keyword within a text document. The goal of the comparison is to study the behavior of the two algorithms with regards to execution time, efficiency and various input cases.

To evaluate the algorithms, the project uses the 20 Newsgroups Dataset, which contains thousands of real-world text documents from different categories such as technology, sports, politics, and science. Real dataset will enable a closer approximation of the algorithms' performance in varying text sizes and search patterns.

The primary goal of this project is to investigate the performance of both algorithms in text searching and classification problems. In addition, the project examines the impact of size of the input and compares the best, average and worst case scenarios, and clarifies why the KMP algorithm is generally more efficient than the Naive algorithm for large text datasets.

1.1 Definition of the Problem

The goal of this project is to find certain patterns in a piece of writing. The goal is to quickly find all the times a pattern appears in a text string. This issue is significant in numerous applications, including search engines and text processing systems.

1.2 Problem Statement

As digital text data grows rapidly, efficient text searching algorithms are indispensable for processing and analyzing vast amount of information efficiently and accurately. Large text documents can be searched slowly with traditional searching methods due to repeated comparisons of characters and ineffective searching procedures.

This project is intended to compare the Naive and KMP searching algorithms to compare their performance in terms of execution time, efficiency, and behavior of the algorithms with different input sizes and different searching cases. The problem we address in this research is to determine which is the better algorithm to search for patterns within large real world text datasets.

1.3 Project Objectives

The primary goals of this project are:

- To learn and understand algorithms for searching text.
- Implement the Naive and KMP algorithms in Python.
- To evaluate the performance of both algorithms on real-world text data.
- To study the execution time of the algorithms for different input sizes.
- To evaluate the best case, average case and worst case.
- To analyze the order of growth and efficiency of each algorithm.
- To find the better algorithm for large text data sets.
- To show the importance of effective searching algorithms in text processing applications.

2. Chosen algorithms

2.1 Naive algorithm

Naive algorithm is one of the simplest algorithms for text search. It matches the pattern against the text at every possible position until it either finds a match or exhausts its search.

2.2 KMP algorithm

Knuth-Morris-Pratt algorithm is an efficient text searching algorithm. It uses the LPS array to avoid unnecessary comparisons during the searching process.

3. Literature Review

The importance of text searching algorithms to information retrieval and text processing systems has led to extensive research on them. Researchers have concentrated on the enhancement of searching efficiency and reduction in execution time in processing large text datasets.

The Naive algorithm is one of the simplest patterns searching algorithms as it compares the pattern with every possible position inside the text. But the algorithm is inefficient for large data set as it keeps comparing characters repeatedly which slows down the algorithm. But the algorithm is very easy to implement and understand.

On the other hand, the Knuth-Morris-Pratt (KMP) algorithm was constructed to optimize the searching process by cutting down on superfluous comparisons. The KMP algorithm employs a pre-processing technique called the Longest Prefix Suffix (LPS) array that allows the algorithm to continue the search without having to start comparisons from the beginning after a mismatch occurs.

The studies have proved that the KMP algorithm is more efficient than the Naive algorithm, especially if there are a large text document and repeated search operations. Due to its improved efficiency, the KMP algorithm is widely used in text editors, search engines, information retrieval systems and cyber security applications.

These studies show the importance of selecting efficient searching algorithms to enhance the system performance and to reduce the processing time in real-world applications.

4. How the algorithms solve the problem

The Naive algorithm looks at every possible position in the text to see if it matches the pattern. This could mean that comparisons happen more than once. The KMP algorithm, on the other hand, speeds up the search process and makes it more efficient by using a preprocessing step (the LPS array) to avoid comparisons that aren't needed.

5. Comparison between chosen algorithms theoretically

Aspect	Naive algorithm	KMP algorithm
Main Objective	Search for a pattern at all possible positions in the text	Efficient pattern search with fewer unnecessary comparisons
Decision made based on	Direct byte-by-byte comparison	LPS array and previously matched information
Basic Concept	Compare pattern against each position in the text	Avoid repeated comparisons by preprocessing
Decision Sequence Production	Sequential Matching from Start to Finish	Continue searching efficiently using LPS array
Approach Used	Simple brute force search	Pattern Search Optimization
Often used in	Learning purposes and small text search applications	search engines, text editors and large text processing systems
Disadvantages	Slow with large datasets due to repeated comparisons	More complex to implement due to preprocessing step.

Table 1: Theoretical comparison of the naive and KMP algorithms.

Section II: Implementation

6. Implementation Setup

Implementation of the current project was done in the python programming language. The naive algorithm as well as the KMP algorithm was developed using Python IDLE.

Various scenarios of inputs have been considered for testing the algorithms.

Component	Specification
Operating System	Windows
Programming Language	Python
IDE/ Tool	Python IDLE
Algorithms	Naive Algorithm and KMP Algorithm

Table 2: Specifications for Experimental Setup

6.1 Programming Language and Tools

Python language was chosen for this project because of its simplicity and convenience to use along with being compatible with text searching algorithms.

These algorithms have been implemented using Python IDLE. The project has used text input as well as pattern matching operations to analyze both Naive and KMP algorithms.

The following tools were used during the implementation:

- Python
- Python IDLE
- Time module for execution time measurement

7. Code Implementation

Both Naive Algorithm and KMP Algorithm have been developed by implementing them in Python code. In each of the algorithms, a separate python file was created and used to test the algorithms on different text data and pattern.

The development involved the following:

- Reading the text inputs
- Searching for patterns within the text
- Calculating the number of patterns matches
- Calculating the execution time
- Comparing both algorithms' performances

In order to compare the two algorithms, they were tested on the same inputs.

8. Dataset Used

In this project we have used 20 Newsgroups Dataset for text classification. 20 Newsgroups Dataset is a collection of text documents from online discussion forums. The dataset is often used for text processing and machine learning experiments because it contains a variety of topics and styles of writing.
The dataset contains categories on sports, technology, politics, science and many more. This variability makes it useful for testing text-searching algorithms on real data, rather than simple, hand-generated examples.
We choose this dataset because it has enough text data for a clear comparison of the performance of Naive and KMP algorithms. Also, it helps us to see how both algorithms behave when working with different lengths of text and different cases of searching.
Another reason for choosing this dataset is its wide use in the research arena of text classification and natural language processing, which makes it a suitable and trustworthy dataset for our experiments.

8.1 Dataset Source

For this project, the data set was downloaded from Kaggle:
https://www.kaggle.com/datasets/crawford/20newsgroups
The dataset consists of about 20,000 text files, which are divided into different categories and discussion topics.

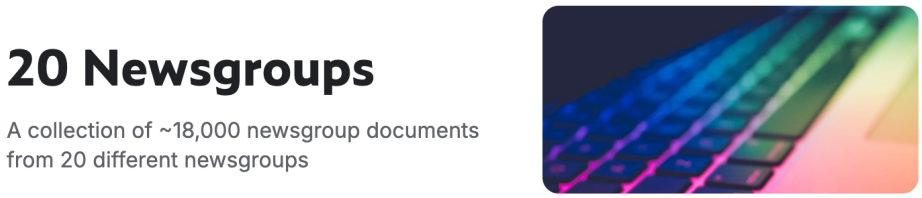

Figure 1: 20 Newsgroups Dataset from Kaggle.

8.2 Categories Used in the Dataset

Category Type	Example Categories
Technology	Comp.graphics, comp.sys.mac.hardware
Sports	rec.sport.baseball, rec.sport.hockey
Politics	talk.politics.misc
Science	sci.space, sci.med
Religion	soc.religion.christian

Table 3: Dataset Categories

The dataset consists of different categories with different topics and writing styles. These categories are used to test the performance of the Naive and KMP algorithms using examples of real text searching from different domains.

9. Input Selection

Different input cases were chosen to test the performance of both algorithms under various conditions. The experiments have been grouped into:

- Best-case
- Average-case
- Worst-case

These input cases are used in comparing algorithms based on execution time, number of comparisons, and search efficiency.
To make a fair and accurate comparison, both algorithms were provided with the same text inputs and patterns.

9.1 Best Case Inputs

The best case is when the pattern is at the beginning of the text. Then the algorithms can find it quickly with less comparisons.

Example 1:
T: "computer science is one of the fastest growing fields in the world."
P: "computer"

Example 2:
T: "network security is essential for protecting data in modern systems."
P: "network"

Example 3:
T: "data analysis plays a key role in decision making processes."
P: "data"

Example 4:
T: "artificial intelligence is transforming many industries today."
P: "artificial"

Example 5 (for KMP):
T: "aaa"

P: "aaa"

9.2 Average Case Inputs

In the average case the pattern occurs somewhere in the middle of the text. This takes a moderate number of comparisons to find the pattern.

Example 1:
T: "many researchers study computer networks to improve communication systems."
P: " network"

Example 2:
T: "the development of artificial intelligence has impacted many fields in technology."
P: " technology"

Example 3:
T: "modern applications rely heavily on data processing and efficient algorithms."
P: "data"

Example 4:
T: "space missions require advanced technology and careful planning to succeed."
P: "technology"

9.3 Worst Case Inputs

In the worst case, the pattern does not exist in the text. Therefore, the algorithms must continue searching through most of the text before stopping.

Example 1:

T: "climate change affects ecosystems and biodiversity across the planet."
P: " " **(No Match)**

Example 2:

T: " space exploration has advanced significantly with missions to mars and the development of new rocket technologies."
P: " " **(No Match)**

Example 3:
T: " the environment is changing rapidly due to climate change and increasing levels of pollution worldwide."

P: " " **(No Match)**

Example 4:
T: " reading books can improve knowledge, enhance vocabulary, and develop critical thinking skills. "

P: " " **(No Match)**

Example 5 (for naive):
T: "aaa"

P: " " **(No Match)**

10. Input Sizes

In the experiments, several input sizes were used in order to evaluate the algorithms' behavior with an increasing amount of text.

Input Size	Description
Small	One short sentence
Medium	One paragraph
Large	Full document from the dataset

Table 4: Input Sizes

To study the execution time and efficiency of both algorithms for larger dataset size, different input sizes are employed.

11. Relationship Between Inputs and Algorithms

The selected input cases were tested using the implemented Python files:

- Naive.py
- KMP.py

The same inputs were used for both algorithms to ensure fairness and accurate comparison during the experiments.

12. Naive Algorithm

12.1 Overview of Naive Algorithm

The Naive algorithm is one of the simplest pattern-searching algorithms, which is used to search for a pattern inside a string. The Naive algorithm works using direct comparison where it compares the pattern with every possible position in the text until a complete match is found. It starts with the first character and compares the pattern characters one by one. If all characters match, the algorithm records one occurrence. If a mismatch occurs, the algorithm shifts the pattern by one position and starts comparing again from the beginning of the pattern.

In this project, the Naive algorithm was implemented to search for keywords inside documents to classify them into categories such as Sports, Technology, Politics, or Mixed/Unknown.

Unlike KMP, the Naive algorithm does not store previous matching information. Therefore, repeated comparisons may occur many times, especially when the text contains repeated characters or partial matches.

Naive algorithm is easy to implement and understand. But it becomes less efficient when processing large text datasets because it may repeatedly compare the same characters.

12.2 Pattern Categories Used in Naive

In the implementation, the patterns were divided into different categories. Each category contains words related to a specific topic.

The Naive algorithm searches for all these patterns inside the input text.

Category	Patterns
Sports	football, game, team, season, player, match
Technology	computer, software, windows, graphics, file, network, data, intelligence, technology, artificial
Politics	government, law, president, policy
Special / Letters	aaa

Table 5: Pattern Categories Used in Naive.

The Sports patterns are used to detect sports-related texts.

The Technology patterns are used to detect technology-related texts.

The Politics patterns are used to detect political texts.

The Special / Letters patterns are included to test repeated characters and compare the behavior of Naive and KMP in special cases.

If the pattern length is zero, the algorithm returns 0 to avoid invalid comparisons and execution errors.

12.3 Naive Search Process

The Naive search process uses two indices:

Index	Meaning
i	Index for the text
j	Index for the pattern

Table 6: Naive algorithm indices.

The algorithm starts by placing the pattern at the beginning of the text.

If:
text[i + j] = pattern[j]

then the comparison continues by increasing j.

If all pattern characters match:

- the occurrence counter increases.

If a mismatch occurs:

- the algorithm shifts the pattern by one position to the right.
- the comparison restarts from the beginning of the pattern.

This process continues until the algorithm reaches the final possible position in the text.

Unlike KMP, the Naive algorithm does not skip comparisons and does not use preprocessing.

12.4 Naive Pseudocode

The following pseudocode explains the main Naive search algorithm.

```
Algorithm Naive_Search(text, pattern)

Input:
 text: the text where the algorithm searches
 pattern: the pattern that should be found

Output:
 count: number of occurrences of the pattern in the text

Convert text and pattern to lowercase

Let n = length of text

Let m = length of pattern

Set count = 0

If m = 0:
 return 0

For i = 0 to n − m:
 Set j = 0

While j < m and text[i + j] = pattern[j]:
 j = j + 1

If j = m:
 count = count + 1

Return count
```

This pseudocode shows that the algorithm compares the pattern with every possible position in the text.

12.5 Naive Code Explanation

The implementation begins by converting both the text and pattern to lowercase which makes the search case-insensitive.

Then the algorithm calculates the lengths of the text and pattern using:

```
n = len(text)
m = len(pattern)
```

If the pattern is empty, the function returns 0. This prevents invalid comparisons and avoids runtime errors.

The algorithm then starts searching using the loop:

```
for i in range(n - m + 1):
```

This loop checks every possible starting position in the text.

Inside the loop, the algorithm compares characters one by one:

```
while j < m and text[i + j] == pattern[j]:
```

If all characters match:

```
if j == m:
    count += 1
```

the algorithm increases the occurrence counter.

Finally, the function returns the total number of matches found in the text.

```
# Patterns for each category
sports_patterns = ["football", "game", "team", "season", "player", "match"]
tech_patterns = ["computer", "software", "windows", "graphics", "file", "network",
"data", "intelligence", "technology", "artificial"]
politics_patterns = ["government", "law", "president", "policy"]
special_patterns = ["aaa"]  # to show the difference between the two algorithms


def naive_search(text, pattern):
    text = text.lower()
    pattern = pattern.lower()
    n = len(text)
    m = len(pattern)
    count = 0
```

```
    if m == 0:
        return 0  # handle empty pattern differently

    for i in range(n - m + 1):
        j = 0
        while j < m and text[i + j] == pattern[j]:
            j += 1
        if j == m:
            count += 1
    return count

def classify_text_naive(text):
    sports_count = sum(naive_search(text, p) for p in sports_patterns)
    tech_count = sum(naive_search(text, p) for p in tech_patterns)
    politics_count = sum(naive_search(text, p) for p in politics_patterns)
    special_count = sum(naive_search(text, p) for p in special_patterns)

    print("Sports count:", sports_count)
    print("Tech count:", tech_count)
    print("Politics count:", politics_count)
    print("Special letters count:", special_count)

    counts = {
        "Sports": sports_count,
        "Technology": tech_count,
        "Politics": politics_count,
        "Special / Letters": special_count
    }
```

```
    max_category = max(counts, key=counts.get)

    if counts[max_category] <= 0:
        return "Mixed / Unknown"
    return max_category

# Allow user input
user_input = input("Enter the text you want to classify: ")
category = classify_text_naive(user_input)
print("Text classification:", category)
```

12.6 Classification Using Naive

The classify_text_naive function applies the Naive search algorithm to all pattern categories.

The following counts are calculated:

```
sports_count = sum(naive_search(text, p) for p in sports_patterns)

tech_count = sum(naive_search(text, p) for p in tech_patterns)

politics_count = sum(naive_search(text, p) for p in politics_patterns)

special_count = sum(naive_search(text, p) for p in special_patterns)
```

After calculating the counts, the function stores them in a dictionary:

```
counts = {
    "Sports": sports_count,
    "Technology": tech_count,
    "Politics": politics_count,
    "Special / Letters": special_count
}
```

Then the category with the highest count is selected using:

```
max_category = max(counts, key=counts.get)
```

If all counts are zero, the function returns: Mixed / Unknown otherwise, it returns the category with the highest number of keyword matches.

13. Naive Time Complexity Analysis

13.1 Complexity of Searching the Text

The searching process may require O(nm) comparisons in many cases because the Naive algorithm searches for the pattern by checking every possible position, so it may compare up to m characters (where m is the length of the pattern) for each position in the text with length n.

13.2 Best Case Time Complexity

In the best case, mismatches occur immediately, or the pattern is found quickly, which makes the algorithm perform fewer comparisons.

Best-case time complexity:

O(n)

13.3 Average Case Time Complexity

In the average case, some characters match while others mismatch, and the algorithm may repeatedly compare many characters.

Average-case complexity:

O(nm)

13.4 Worst Case Time Complexity

The worst case occurs when the text contains repeated characters or when the pattern partially matches many times before failing.
Example:

Text:
aaaaaaaaaa

Pattern:
aaaab

In this case, the algorithm repeatedly compares almost the entire pattern before finding a mismatch.

Worst-case complexity:

O(nm)

14. Naive Experimental Results

14.1 Timing Mechanism

The execution time of the Naive algorithm was measured using Python's time module. The timer starts before calling the classification function and stops after the function returns the predicted category.

The following code was used:

```
start = time.time()
category = classify_text_naive(user_input)
end = time.time()

print("Naive Execution Time:", end - start, "seconds")
```

The execution time is calculated as:

Execution Time = End Time − Start Time

The result is reported in seconds. In the tables, the values were also converted to milliseconds for easier comparison.

14.2 Naive Execution Time for Best Case

The best case represents inputs where the searched pattern appears clearly in the text or the mismatch occurs quickly.

Case Type	Example	Naive Time (sec)	Naive Time (ms)
Best Case	1	0.03373	33.73
Best Case	2	0.03015	30.15
Best Case	3	0.03569	35.69
Best Case	4	0.03597	35.97
Best Case	5	0.05002	50.02

Table 7: Naive Execution Time for Best Case.

The average best-case time is calculated as:

Average Best-case Time= 0.03711 seconds

Average Best-case Time = 37.11 milliseconds

14.3 Naive Execution Time for Average Case

The average case represents inputs where the pattern appears in the middle of the text or where the search includes a normal number of comparisons.

Case Type	Example	Naive Time (sec)	Naive Time (ms)
Average Case	1	0.03139	31.39
Average Case	2	0.03351	33.51
Average Case	3	0.03441	34.41
Average Case	4	0.03499	34.99

Table 8: Naive Execution Time for Average Case.

The average average-case time is calculated as:

Average = 0.03358 seconds

Average = 33.58 milliseconds

14.4 Naive Execution Time for Worst Case

The worst case represents inputs where the pattern does not exist or where repeated partial matches occur.

Case Type	Example	Naive Time (sec)	Naive Time (ms)
Worst Case	1	0.02961	29.61
Worst Case	2	0.03191	31.91
Worst Case	3	0.03678	36.78
Worst Case	4	0.03521	35.21
Worst Case	5	0.05002	50.02

Table 9: Naive Execution Time for Worst Case.

The average worst-case time is calculated as:

Average Worst-case Time= 0.03671 seconds

Average Worst-case Time = 36.71 milliseconds

14.5 Overall Naive Average Execution Time

To calculate the overall average execution time, all Naive execution times were added together and divided by the total number of examples.

The total number of examples is:

5 Best Case + 4 Average Case + 5 Worst Case = 14 examples

The total execution time is:

0.18556 + 0.13430 + 0.18353 = 0.50339 seconds

The overall average is:

Overall Average=0.50339/14Overall\ Average = 0.50339 / 14Overall Average=0.50339/14

0.5033914=0.03596\frac{0.50339}{14}=0.03596140.50339 =0.03596

Overall Average = 0.03596 seconds

Overall Average = 35.96 milliseconds

14.6 Summary Table of Naive Average Results

Case Type	Number of Examples	Total Time (sec)	Average Time (sec)	Average Time (ms)
Best Case	5	0.18556	0.03711	37.11
Average Case	4	0.13430	0.03358	33.58
Worst Case	5	0.18353	0.03671	36.71
Overall	14	0.50339	0.03596	35.96

Table 10: Naive Average Results.

14.7 Discussion of Experimental Results

The results show that the Naive algorithm works correctly for different input cases. The average execution time for the best case was 0.03711 seconds, the average-case time was 0.03358 seconds, and the worst-case time was 0.03671 seconds.

Although the execution times are relatively close for these small and medium-sized inputs, the theoretical analysis shows that the Naive algorithm becomes less efficient for larger datasets because of repeated comparisons.

The Naive algorithm does not use preprocessing or optimization techniques. Therefore, repeated characters and partial matches may increase the running time significantly.

Compared to KMP, the Naive algorithm performs more unnecessary comparisons, especially in worst-case scenarios. As the dataset size grows, KMP is expected to perform better because of its linear order of growth.

Analysis Point	**Naive Algorithm**
Algorithm Name	Naive String Matching
Algorithm Type	String Matching
Main Technique	Direct Character Comparison
Best Case Time Complexity	O(n)
Average Case Time Complexity	O(nm)
Worst Case Time Complexity	O(nm)
Space Complexity	O(1)
Extra Memory Used	None
Main Advantage	Simple Implementation
Main Limitation	Repeated Comparisons
Growth Type	Quadratic Growth

Table 11: Naive Complexity.

15. Naive Output Results

15.1 Best Case Output

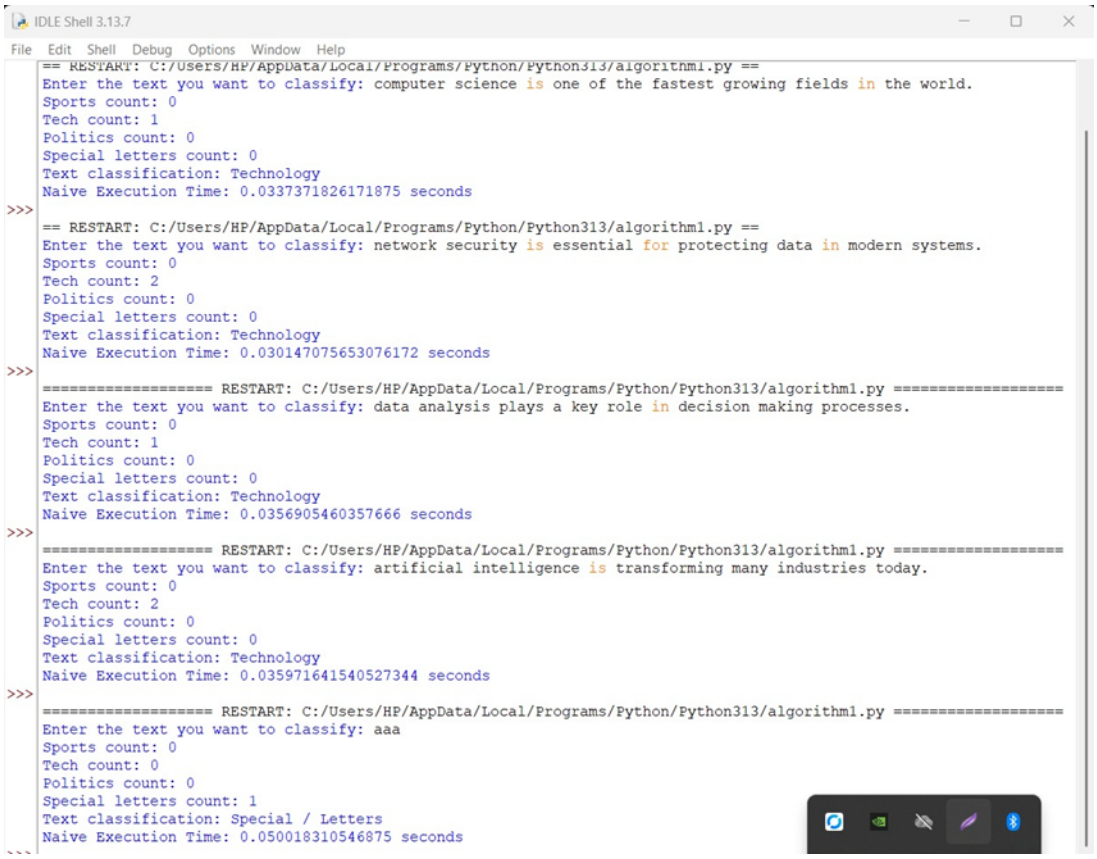

Figure 2: Best case execution results for the Naive algorithm.

15.2 Average Case Output

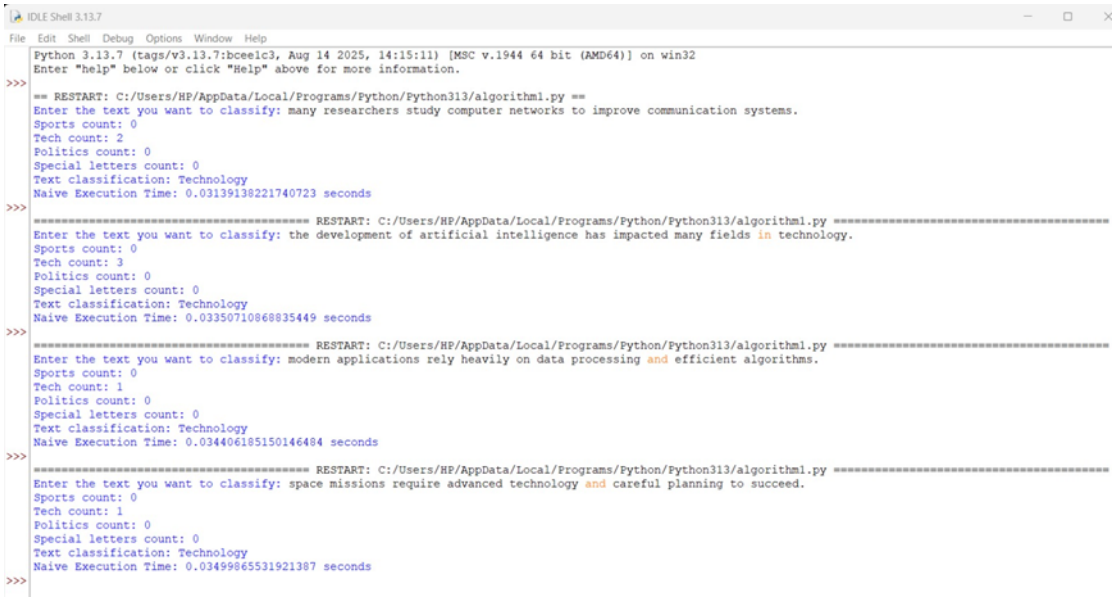

Figure 3: Average case execution results for the Naive algorithm.

15.3 Worst Case Output

Figure 4: Worst case execution results for the Naive algorithm.

16. KMP Algorithm

16.1 Overview of KMP Algorithm

KMP (Knuth-Morris-Pratt) algorithm is highly ranked among string matching algorithms. Its main function being a tool to locate a certain pattern within a piece of text. In our project, we implemented KMP algorithm to identify the presence of certain keywords in text documents which will help us categorize the texts under categories such as Sports, Technology, Politics, or Mixed / Unknown.

KMP algorithm is based on a very simple observation that enables it to skip certain texts while matching. For instance, while matching with the "Naive" approach, a mismatch usually leads to shifting the pattern by one position and then starting the comparison of characters from the beginning of the pattern. This causes a lot of repetitive comparison especially when the text or the pattern contains the same characters repeating several times. However, KMP is able to cut down this time by making use of the LPS array, which is a result of pattern preprocessing. The LPS array stores the information about how far the pattern can be shifted even if a mismatch occurs.

Thus, KMP is the algorithm that should be preferred when working with large text datasets since it cuts down the number of comparisons and leaves the process of searching with maximum efficiency. Thanks to that it is the main component of certain tools we use daily like search engines, text editors, plagiarism detection systems, DNA sequence matching programs, and document classification systems.

16.2 Purpose of Using KMP in This Project

The main task for the project is to find particular keywords inside text documents and afterwards classify each text on the basis of how many keywords have been matched. The dataset has several types of text documents randomly put together, and the system is to decide whether a text belongs to sports, technology, politics, or none of these categories.

On top of that, there is another main reason for choosing KMP over other existing string matching algorithms i.e. extensive use of pattern matching that leads to inefficiency. Therefore, in this project each keyword was a pattern, and the main text was the rest of the document apart from the pattern. The algorithm first extracts every single keyword from the document and then counts its appearances in the document.

Let's suppose that the keywords computer, network, or data get the Technology category, whereas football, team, or match correspond to Sports. Once all the keywords have been searched, the category holding the highest count is decided as the predicted category.

This method gives the project a chance to carry out keyword-based text classification with a simple, rule-based, and algorithmic method.

16.3 Pattern Categories Used in KMP

In the implementation, the patterns were divided into different categories. Each category contains a group of words related to a specific topic. The KMP algorithm searches for all these patterns inside the input text.

The pattern categories used in the project are shown below:

Category	Patterns
Sports	football, game, team, season, player, match
Technology	computer, software, windows, graphics, file, network, data, intelligence, technology, artificial
Politics	government, law, president, policy
Special / Letters	aaa, empty pattern

Table 12: Pattern Categories Used in KMP Algorithm.

The Sports patterns are used to detect sports-related texts. The Technology patterns are used to detect technology-related texts. The Politics patterns are used to detect political texts. The Special / Letters patterns are added to test special cases such as overlapping characters and empty patterns.

The empty pattern is handled separately in the code. If the pattern length is zero, the algorithm returns 0. This avoids errors during execution and makes the algorithm safer.

16.4 General Idea of How KMP Works

The KMP algorithm operates in two significant phases. In the first phase, the algorithm performs preprocessing to create the LPS array for the given pattern. In the second phase

the algorithm employs the LPS array to locate the pattern inside the text.

Two pointers are kept during the searching time. One pointer steps through the text while the other steps through the pattern. Both pointers are incremented if the matching characters are found. When there is a mismatch, KMP does not shift the text pointer back. Rather, it relies on the LPS array to find the new position of the pattern pointer.

This gives KMP a big leg up. It doesn't unnecessarily go over the same comparisons again and again, save them for the initial ones only, so to speak. This means that compared to the Naive algorithm, the search time reduces significantly and the efficiency increases.

16.5 LPS Array Explanation

In fact, the LPS array is the Longest Prefix Suffix array. It is an aid array that the KMP algorithm employs to keep track of the pattern. The LPS value at any index specifies the length of the longest proper prefix which is also a suffix to the substring ending at that index.

A prefix is the portion of the pattern starting from the beginning. On the other hand, a suffix is the portion of the pattern ending at the current position. The full string itself is excluded when considering a proper prefix.

Consider the pattern: [aaa]

The LPS array

for this pattern is: [0, 1, 2]

This indicates that the proper prefix and suffix do not exist at the first character. The longest prefix which is also a suffix turns out to be the one character 'a' at the second character, so the value is set as 1. At the last character, the longest prefix which is also a suffix is the two characters 'aa', so the value is 2.

The LPS array helps the algorithm to find out from which point it should resume after a mismatch. Rather than testing the pattern from its start repeatedly, KMP tries to take advantage of the previously matched part in order to make a jump to a more promising place.

16.6 Example of LPS Array Construction

To understand the LPS array more clearly, consider the pattern:[aaa]

The algorithm starts by initializing the LPS array with zeros: [0, 0, 0]

The first character always has an LPS value of 0 because there is no proper prefix and suffix for one character.

Then the algorithm compares the next characters:

Step	i	Pattern[i]	length	Result
Start	1	a	0	Compare pattern[1] with pattern[0]
Match	1	a	1	LPS[1] = 1
Next	2	a	1	Compare pattern[2] with pattern[1]
Match	2	a	2	LPS[2] = 2

Table 13: Step-by-Step Construction of LPS Array for Pattern .

So the final LPS array becomes: [0, 1, 2]

This example shows how KMP handles repeated characters efficiently. If a mismatch occurs after matching part of the pattern, the algorithm uses this LPS information to avoid restarting the search.

16.7 KMP Search Process

After building the LPS array, the algorithm starts searching for the pattern inside the text. The search process uses two indices:

Index	Meaning
i	Index for the text
j	Index for the pattern

Table 14: Meaning of KMP Search Indices .

If text[i] matches pattern[j], both indices increase by one. If j reaches the length of the pattern, this means that a full match has been found, so the count is increased by one.

After finding a match, KMP does not stop. It continues searching for more occurrences. This is useful because the same keyword may appear more than once in the text.

If a mismatch occurs, the algorithm checks the value of j. If j is not zero, it updates j using the LPS array: j = lps[j - 1]

If j is zero, the algorithm moves to the next character in the text by increasing i.

This process continues until the whole text has been scanned.

16.8 KMP Pseudocode

The following pseudocode explains the main KMP search algorithm.

```
Algorithm KMP_Search(text, pattern)

Input:
    text: the text where the algorithm searches
    pattern: the pattern that should be found

Output:
    count: number of occurrences of the pattern in the text

 Convert text and pattern to lowercase
  Let n = length of text
  Let m = length of pattern
  Set count = 0
```

```
If m = 0:
    return 0

Build the LPS array for the pattern

  Set i = 0
 Set j = 0

  While i < n:
     If pattern[j] = text[i]:
         i = i + 1
         j = j + 1

     If j = m:
         count = count + 1
         j = LPS[j - 1]

     Else if i < n and pattern[j] != text[i]:
         If j != 0:
             j = LPS[j - 1]
         Else:
             i = i + 1

 Return count
```

This pseudocode shows that the text pointer does not move backward. This is one of the main reasons why KMP is efficient.

16.9 LPS Pseudocode

The following pseudocode explains how the LPS array is built.

```
Algorithm Build_LPS(pattern)

Input:
    pattern: the pattern to be searched

Output:
    LPS array

Let m = length of pattern
Create LPS array of size m and initialize all values to 0
  Set length = 0
  Set i = 1

    While i < m:
```

```
        If pattern[i] = pattern[length]:
            length = length + 1
            LPS[i] = length
            i = i + 1

        Else:
            If length != 0:
                length = LPS[length - 1]
            Else:
                LPS[i] = 0
                i = i + 1

    Return LPS
```

The LPS construction requires only one scan of the pattern. This makes the preprocessing step efficient.

16.10 KMP Code Explanation

The implementation starts by converting both the text and the pattern to lowercase. This makes the search case-insensitive, meaning that words such as Computer, computer, and COMPUTER are treated as the same word.

Then the algorithm calculates the lengths of the text and pattern using:

```
n = len(text)
 m = len(pattern)
```

If the pattern is empty, the function returns 0. This is done to prevent errors because searching for an empty pattern can cause an invalid access to pattern characters.

After that, the algorithm builds the LPS array. The LPS array is initialized with zeros:

```
lps = [0] * m
```

Then the algorithm fills the LPS array using a loop. If the current character matches the character at the current prefix length, the length increases and the LPS value is updated. If the characters do not match, the algorithm uses previous LPS values to adjust the length.

After building the LPS array, the algorithm starts the actual search. It compares the text and pattern characters using two pointers. When a full pattern is found, the count increases. When a mismatch occurs, the LPS array is used to decide the next position in the pattern.

Finally, the function returns the total number of occurrences of the pattern in the text.

```
# Patterns for each category

sports_patterns = ["football", "game", "team", "season", "player", "match"]

tech_patterns = ["computer", "software", "windows", "graphics", "file", "network", "data",
"intelligence", "technology", "artificial"]

politics_patterns = ["government", "law", "president", "policy"]


# Special letters patterns (new, cause differences)

special_patterns = ["aaa", ""]  # overlapping and empty


def kmp_search(text, pattern):

    # Convert to lowercase for case-insensitive matching

    text = text.lower()

    pattern = pattern.lower()

    n = len(text)

    m = len(pattern)

    count = 0


    if m == 0:

        return 0  # handle empty pattern differently


    # Build LPS (Longest Prefix Suffix) array

    lps = [0] * m

    length = 0

    i = 1

    while i < m:

        if pattern[i] == pattern[length]:

            length += 1
```

```
            lps[i] = length
            i += 1
        else:
            if length != 0:
                length = lps[length - 1]
            else:
                lps[i] = 0
                i += 1

    # KMP search
    i = 0
    j = 0
    while i < n:
        if pattern[j] == text[i]:
            i += 1
            j += 1
        if j == m:
            count += 1
            j = lps[j - 1]
        elif i < n and pattern[j] != text[i]:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return count
```

```
def classify_text_kmp(text):
    # Count occurrences for each category
    sports_count = sum(kmp_search(text, p) for p in sports_patterns)
    tech_count = sum(kmp_search(text, p) for p in tech_patterns)
    politics_count = sum(kmp_search(text, p) for p in politics_patterns)
    special_count = sum(kmp_search(text, p) for p in special_patterns)

    # Print counts for debugging
    print("Sports count:", sports_count)
    print("Tech count:", tech_count)
    print("Politics count:", politics_count)
    print("Special letters count:", special_count)

    # Determine category with highest count including special patterns
    counts = {
        "Sports": sports_count,
        "Technology": tech_count,
        "Politics": politics_count,
        "Special / Letters": special_count
    }
    max_category = max(counts, key=counts.get)
    # If the highest count is 0, no match at all
    if counts[max_category] == 0:
        return "Mixed / Unknown"
    return max_category
# Allow user input
```

```
user_input = input("Enter the text you want to classify: ")

category = classify_text_kmp(user_input)

print("Text classification:", category)
```

16.11 Classification Using KMP

The classify_text_kmp function applies the KMP search function to all pattern categories. It calculates the total number of matches for each category.

The following counts are calculated:

```
sports_count = sum(kmp_search(text, p) for p in sports_patterns)
 tech_count = sum(kmp_search(text, p) for p in tech_patterns)
 politics_count = sum(kmp_search(text, p) for p in politics_patterns)
 special_count = sum(kmp_search(text, p) for p in special_patterns)
```

After calculating the counts, the function stores them in a dictionary:

```
counts = {
    "Sports": sports_count,
    "Technology": tech_count,
    "Politics": politics_count,
    "Special / Letters": special_count
 }
```

Then the category with the highest count is selected using:

```
max_category = max(counts, key=counts.get)
```

If all counts are zero, the function returns:

Mixed / Unknown

Otherwise, it returns the category with the highest number of keyword matches.

17. KMP Time Complexity Analysis

17.1 Complexity of Building the LPS Array

The first part of KMP is building the LPS array. In this step, the algorithm scans the pattern and computes the longest prefix-suffix values.

If the length of the pattern is m, then the LPS construction takes linear time: O(m)

This is because the algorithm processes each character of the pattern without using nested loops that repeat the full pattern many times.

17.2 Complexity of Searching the Text

The second part is searching the pattern inside the text. If the length of the text is n, the search process takes: O(n)

The reason is that the text pointer moves only forward. Even when a mismatch occurs, the algorithm does not move the text pointer backward. Instead, it adjusts the pattern pointer using the LPS array.

This means that each character in the text is processed efficiently, and the algorithm avoids repeated comparisons.

17.3 Total Time Complexity

The total time complexity of KMP is the sum of the preprocessing time and the searching time. T(n, m) = O(m) + O(n)

Therefore: T(n, m) = O(n + m)

Where:

Symbol	Meaning
n	Length of the text
m	Length of the pattern

Table 15: Symbols Used in KMP Time Complexity Analysis.

17.4 Best Case Time Complexity

In the best case, the pattern may be found near the beginning of the text, or the matching process may occur with few mismatches. However, KMP still builds the LPS array and scans the text.

Therefore, the best-case time complexity is: O(n + m)

Even though the actual running time may be low, the theoretical order remains linear.

17.5 Average Case Time Complexity

In the average case, the pattern may appear somewhere in the middle of the text, or there may be some mismatches during searching. KMP handles these mismatches efficiently using the LPS array.

The average-case time complexity is: O(n + m)

This shows that KMP maintains stable performance even when the input text varies.

17.6 Worst Case Time Complexity

In the worst case, the text may contain many partial matches before mismatches occur. In the Naive algorithm, this can cause many repeated comparisons. However, KMP avoids this issue by using the LPS array.

Therefore, even in the worst case, KMP has a time complexity of: O(n + m)

This is one of the strongest advantages of the KMP algorithm.

18. KMP Experimental Results

18.1 Timing Mechanism

The execution time of the KMP algorithm was measured using Python's time module. The timer starts before calling the classification function and stops after the function returns the predicted category.

The following code was used:

```
start = time.time()
 category = classify_text_kmp(user_input)
 end = time.time()

print("KMP Execution Time:", end - start, "seconds")
```

The execution time is calculated as:

Execution Time = End Time - Start Time

The result is reported in seconds. In the table, the values were also converted to milliseconds for clearer comparison.

18.2 KMP Execution Time for Best Case

The best case represents inputs where the searched pattern appears clearly in the text or the match is found with fewer unnecessary comparisons.

Case Type	Example	KMP Time (sec)	KMP Time (ms)
Best Case	1	0.02961	29.61
Best Case	2	0.03410	34.10
Best Case	3	0.03302	33.02
Best Case	4	0.04929	49.29
Best Case	5	0.07451	74.51

Table 16: KMP Execution Time Results for Best Case Inputs.

The average best-case time is calculated as:

Average = (0.02961 + 0.03410 + 0.03302 + 0.04929 + 0.07451) / 5

Average = 0.04411 seconds

Average = 44.11 milliseconds

18.3 KMP Execution Time for Average Case

The average case represents inputs where the pattern appears in the middle of the text or where the search process includes a normal number of comparisons and mismatches.

Case Type	Example	KMP Time (sec)	KMP Time (ms)
Average Case	1	0.03229	32.29
Average Case	2	0.03153	31.53
Average Case	3	0.03188	31.88
Average Case	4	0.04893	48.93

Table 17: KMP Execution Time Results for Average Case Inputs

The average average-case time is calculated as:

Average = (0.03229 + 0.03153 + 0.03188 + 0.04893) / 4

Average = 0.03616 seconds

Average = 36.16 milliseconds

18.4 KMP Execution Time for Worst Case

The worst case represents inputs where the pattern does not exist or where there are repeated partial matches. These cases are usually more difficult for the Naive algorithm, but KMP handles them efficiently because of the LPS array.

Case Type	Example	KMP Time (sec)	KMP Time (ms)
Worst Case	1	0.03013	30.13
Worst Case	2	0.03284	32.84
Worst Case	3	0.03366	33.66
Worst Case	4	0.04423	44.23
Worst Case	5	0.05806	58.06

Table 18: KMP Execution Time Results for Worst Case Inputs.

The average worst-case time is calculated as:

Average = (0.03013 + 0.03284 + 0.03366 + 0.04423 + 0.05806) / 5

Average = 0.03978 seconds

Average = 39.78 milliseconds

18.5 Overall KMP Average Execution Time

To calculate the overall average execution time, all KMP execution times were added together and divided by the total number of examples.

The total number of examples is:

5 Best Case + 4 Average Case + 5 Worst Case = 14 examples

The total execution time is:

0.22053 + 0.14463 + 0.19892 = 0.56408 seconds

The overall average is:

Overall Average = 0.56408 / 14

Overall Average = 0.04029 seconds

Overall Average = 40.29 milliseconds

18.6 Summary Table of KMP Average Results

Case Type	#Examples	Total Time (sec)	Average Time (sec)
Best Case	5	0.22053	0.04411
Average Case	4	0.14463	0.03616
Worst Case	5	0.19892	0.03978
Overall	14	0.56408	0.04029

Table 19: Summary of KMP Average Execution Time Results

18.7 Discussion of Experimental Results

The results show that the KMP algorithm has stable execution time across different input cases. The average execution time for the best case was 0.04411 seconds, the average-case time was 0.03616 seconds, and the worst-case time was 0.03978 seconds.

Although the best case has a slightly higher average than the average and worst cases in these results, this does not mean that the best case is theoretically slower. The difference can be caused by factors such as text length, number of searched patterns, system performance, and background processes running on the computer during execution.

From the theoretical side, KMP is efficient because its time complexity remains O(n + m) in all cases. This means that the running time grows linearly with the size of the text and the pattern. As the dataset becomes larger, KMP is expected to perform better than the Naive algorithm, especially when there are repeated characters or partial matches.

19. KMP Analysis Summary

19.1 Final KMP Complexity Table

Analysis Point	KMP Algorithm
Algorithm Name	Knuth-Morris-Pratt
Algorithm Type	String Matching
Main Technique	LPS preprocessing
Best Case Time Complexity	O(n + m)
Average Case Time Complexity	O(n + m)
Worst Case Time Complexity	O(n + m)
Space Complexity	O(m)
Extra Memory Used	LPS array
Main Advantage	Avoids repeated comparisons
Main Limitation	Requires preprocessing and extra space

Table 20: Final Complexity Summary of KMP Algorithm .

20. KMP Output Results

20.1 Best Case Output

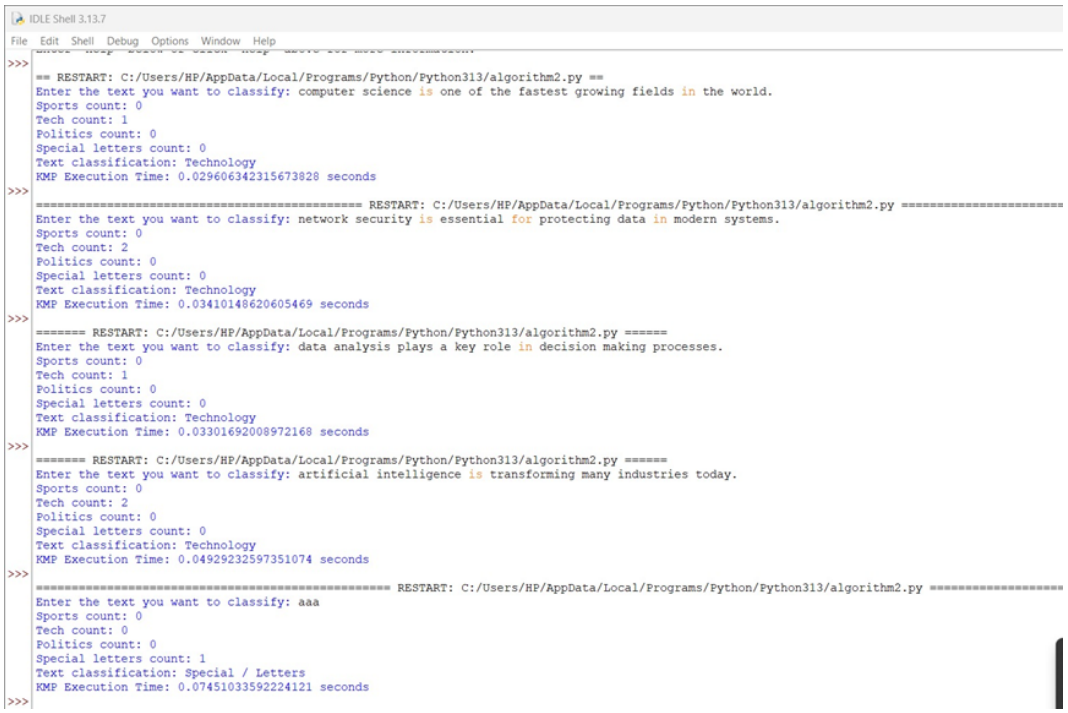

Figure 5: Best case execution results for the KMP algorithm.

20.2 Average Case Output

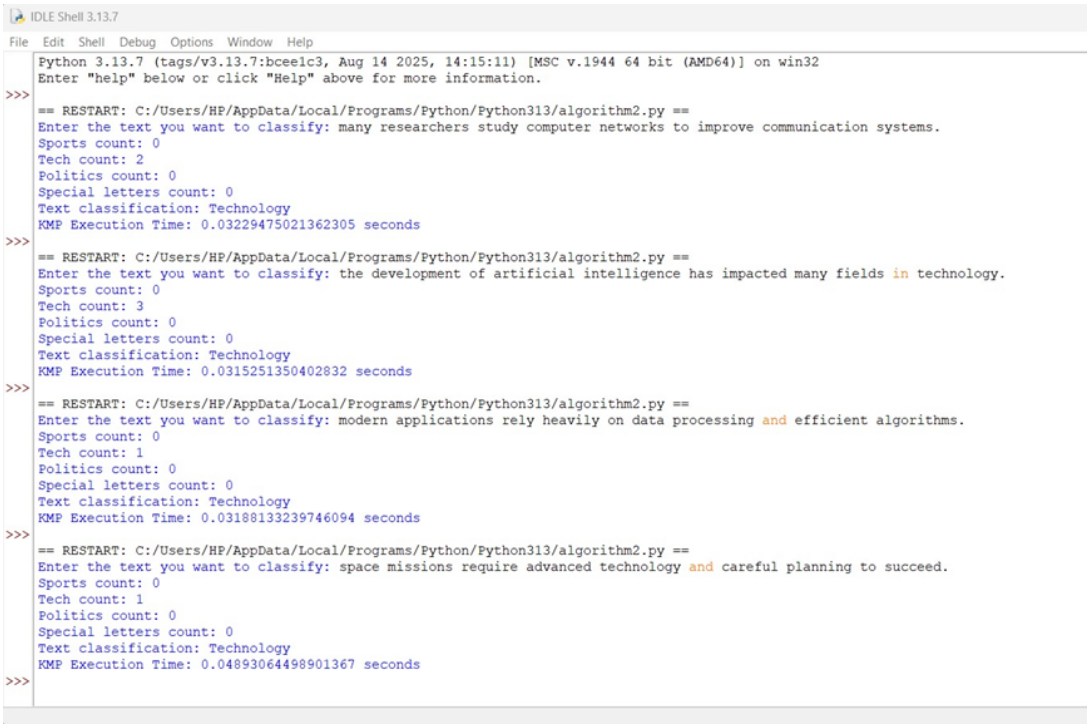

Figure 6: Average case execution results for the KMP algorithm.

20.3 Worst Case Output

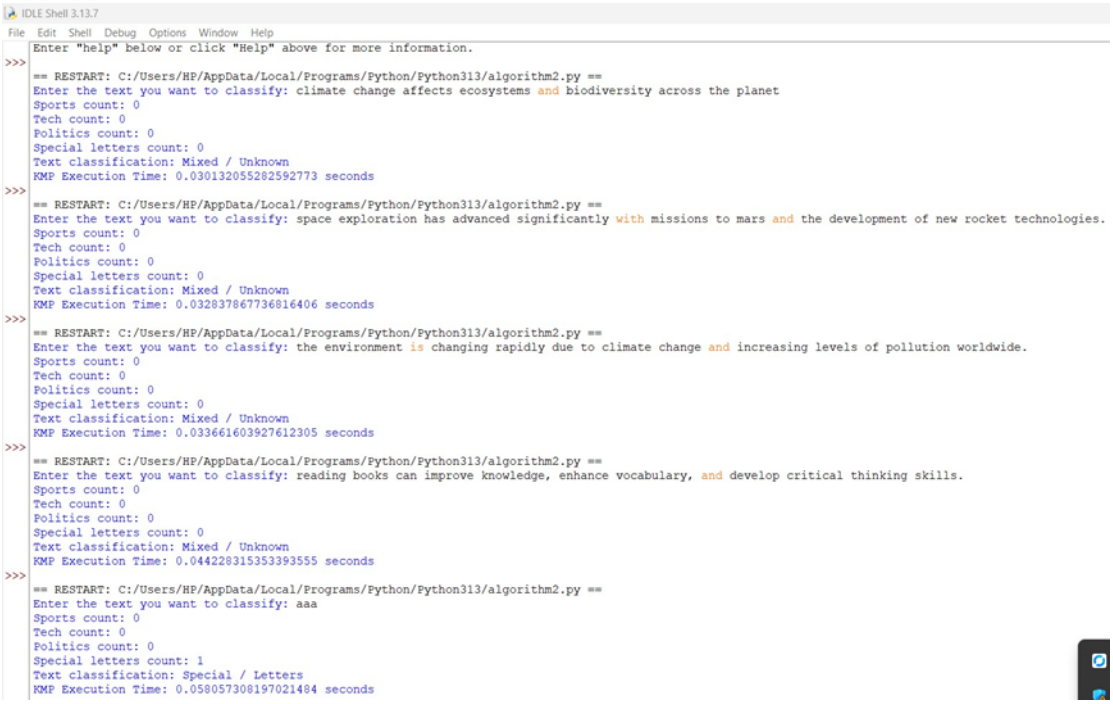

Figure 7: Worst case execution results for the KMP algorithm.

Section III: Analysis and Experimental Results

21. Space Complexity Analysis

21.1 Overview of Space Complexity

Space complexity is a vital aspect of algorithm analysis as it quantifies how much additional memory an algorithm requires while executing. This project involves using the Naive string matching algorithm and KMP algorithm to search patterns within text documents. So, it makes perfect sense to compare the time they take for execution as well as their memory consumption.

Typically, the memory consumed by a string matching algorithm can be segregated into two portions. Primarily, the memory that holds the input text along with the pattern. This memory is obligatory for both the algorithms as they both Naive and KMP need the text and the pattern at hand to carry out the searching. Secondly, the supplementary memory used internally by the algorithm at the time of execution. This portion is generally what is taken into account while analyzing the space complexity.

Within this project, the variable text stands for the entire text, whereas the variable pattern stands for the word to be found. The length of the text is denoted by n, and the length of the pattern is denoted by m.

Symbol	Meaning
n	Length of the input text
m	Length of the pattern
k	Number of patterns used for classification

Table 21: Symbols Used in Space Complexity Analysis.

21.2 Space Complexity of the Naive Algorithm

Naive algorithm relies on a very straightforward method to locate a pattern within a text. It goes through every potential starting position in the text and performs a character-by-character comparison between the pattern and the text.

Regarding memory consumption, the Naive algorithm does not generate any additional array or data structure that depends on the pattern length or text length. It just uses a few variables like:

```
n = len(text)
 m = len(pattern)
 count = 0
 i
 j
```

Those variables hold the length of the text, the pattern length, the number of

matches, and the indices of the looping. Since the count of these variables doesn't go up with the input size they are considered constant memory.

This is why the additional space complexity of the Naive algorithm is: `O(1)`

This means that the Naive algorithm uses constant extra memory. Even if the text becomes larger or the pattern becomes longer, the algorithm still uses almost the same amount of additional memory.

21.3 Memory Consumption in the Naive Algorithm

Because it does not require any preprocessing, the naive algorithm is very memory friendly.

Indeed, it will not construct any array before searching. It just directly compares the pattern with the text.

For instance, the naive algorithm in the project code changes the text and pattern to lowercase, then utilizes nested loops for character comparison. Memory is mainly taken up by the input text, the pattern, and several elementary variables.

The Naive algorithm uses memory for:

Memory Item	Purpose	Space
text	Stores the input text	O(n) input storage
pattern	Stores the searched pattern	O(m) input storage
count	Counts pattern occurrences	O(1)
i, j	Loop counters	O(1)
Extra array	Not used	None

Table 22: Memory Consumption in the Naive Algorithm.

When analyzing extra space only, the Naive algorithm is considered: `O(1)`

This is because the input text and pattern are already given to the algorithm, and the algorithm does not create a new structure that grows with input size.

21.4 Space Complexity of the KMP Algorithm

The KMP algorithm, since it implements a preprocessing operation before the actual search, is classified as a more advanced method compared to the Naive algorithm. This preprocessing step is about creating an additional array known as the LPS array (which means Longest Prefix Suffix).

The LPS array in the project's source code is created by the following statement:

lps = [0] * m

This is to say that the LPS array's length is determined by the pattern's length. So if pattern length is m, then the LPS array also contains m elements.

Having this array, KMP also needs some extra memory. So, its space complexity will be: O(m)

Where m denotes the length of the pattern.

This additional memory serves to keep the pattern's information, allowing the algorithm to get rid of unnecessary repeated comparisons during the search phase.

21.5 Memory Consumption in the KMP Algorithm

The KMP algorithm consumes memory for the same main variables that Naive algorithm uses, for example, text, pattern, n, m, count, i, and j. On the other hand, KMP needs additional memory for the LPS array.

The LPS array keeps the length of the longest prefix which is also a suffix for each location in the pattern. This data enables the algorithm to know where to resume searching after a mismatch.

The memory used by KMP can be summarized as follows:

Memory Item	Purpose	Space
text	Stores the input text	O(n) input storage
pattern	Stores the searched pattern	O(m) input storage
lps	Stores prefix-suffix values	O(m)
count	Counts pattern occurrences	O(1)
i, j	Text and pattern pointers	O(1)
length	Tracks current prefix length	O(1)

Table 23: Memory Consumption in the KMP Algorithm .

When analyzing the additional memory used by the algorithm, the most important part is the LPS array. Since its size grows with the pattern length, the extra space complexity of KMP is: O(m)

21.6 Why KMP Needs Extra Memory

KMP needs extra memory because it creates the LPS array before searching. This array is not used in the Naive algorithm.

The objective of the LPS array is to record valuable data about the pattern. If a mismatch happens while performing the search, KMP does not go back to the starting of the pattern. Rather, it refers to the LPS value to determine the next position of the pattern pointer.

For instance, in case the pattern is: [aaa]

The LPS array is:[0, 1, 2]

This means that the algorithm knows that part of the pattern has already been matched before. If a mismatch happens, KMP can continue from a better position instead of repeating all comparisons.

This is why KMP uses more memory than Naive. However, this extra memory improves the algorithm's time performance. In other words, KMP trades a small amount of additional memory for better searching efficiency.

21.7 Example Explaining KMP Memory Usage

Consider the pattern: [aaa]

The length of the pattern is: [m = 3]

KMP creates an LPS array of size 3: lps = [0, 1, 2]

This means the algorithm stores three integer values in memory. If the pattern length becomes 10, the LPS array will contain 10 values. If the pattern length becomes 100, the LPS array will contain 100 values.

Therefore, the memory required by the LPS array grows linearly with the pattern length. This is why the space complexity of KMP is: O(m)

The larger the pattern, the larger the LPS array. However, the memory growth is still linear and acceptable for most text searching applications.

21.8 Comparison Between Naive and KMP in Space Complexity

The main difference between Naive and KMP in terms of memory usage is that Naive does not require preprocessing, while KMP requires the LPS array.

Algorithm	Extra Data Structure	Extra Space Complexity	Explanation
Naive	None	O(1)	Uses only counters and loop variables
KMP	LPS array	O(m)	Stores prefix-suffix values for the pattern

Table 24: Space Complexity Comparison Between Naive and KMP Algorithms.

The Naive algorithm is better in terms of memory consumption because it uses constant extra memory. However, it may take more time because it repeats comparisons.

The KMP algorithm uses more memory because of the LPS array. However, this additional memory allows it to reduce repeated comparisons and improve the search process, especially for larger texts or repeated patterns.

21.9 Space Complexity in the Project Implementation

In this project, both algorithms are used for keyword-based text classification. This means that each algorithm searches for several patterns from different categories such as Sports, Technology, Politics, and Special / Letters.

For the Naive algorithm, each pattern is searched directly without creating any additional array. Therefore, the extra memory remains constant for each search:

Naive Space Complexity = O(1)

For the KMP algorithm, an LPS array is created for each searched pattern. If the current pattern length is m, then the LPS array size is m:

KMP Space Complexity = O(m)

Since the project searches multiple keywords, KMP builds a separate LPS array for each keyword during its search. However, after the search for one keyword finishes, the LPS array can be released from memory. Therefore, the maximum extra memory at one time depends on the longest pattern, not on the total number of patterns.

For example, if the longest keyword is: intelligence

then the largest LPS array size depends on the length of this word.

21.10 Memory Trade-off Between Naive and KMP

The Naive algorithm and KMP entail a trade-off. While the Naive algorithm consumes less memory, it may be slower since it starts comparing the pattern characters all over again after mismatches.

On the other hand, KMP allocates additional memory to store the LPS array. However, this memory is very useful for avoiding repeated comparisons. That is why KMP, in terms of time complexity, especially in average and worst-case, is more efficient.

The trade-off can be summarized as follows:

Algorithm	Memory Usage	Time Efficiency	Main Advantage
Naive	Lower memory	Slower in worst case	Simple and memory-efficient
KMP	Higher memory due to LPS	Faster and more stable	Avoids repeated comparisons

Table 25: Memory and Time Trade-off Between Naive and KMP Algorithms.

So, if memory is the major aspect and the text is pretty short, going with the Naive approach might be okay. But on the other hand, if the text is very long and finding the pattern quickly matters, then KMP generally does better as the additional memory used for the LPS array helps speeding up the performance.

21.11 Final Space Complexity Summary

Algorithm	Best Case Space	Average Case Space	Worst Case Space	Reason
Naive	O(1)	O(1)	O(1)	Does not use extra array
KMP	O(m)	O(m)	O(m)	Uses LPS array of size m

Table 26: Final Space Complexity Summary of Naive and KMP Algorithms.

Naive algorithm's space complexity is constant for all different cases because it only uses simple variables and does not keep extra information on the pattern. In contrast, the space complexity of KMP is directly proportional to the length of the pattern since it constructs the LPS array even before the searching.

Though KMP consumes more memory, the extra memory is effectively utilized as it leads to a faster search operation. The LPS array enables KMP to skip unnecessary repeated comparisons, thereby making the algorithm more efficient on bigger datasets.

22. Order of Growth

The order of growth (represented with Big-O notation) is used to describe how the running time of an algorithm increases as the input size becomes larger, which helps measure the efficiency of algorithms when dealing with large datasets.

22.1 Naive Algorithm Order of Growth

- Best Case: O(n)
- Average Case: O(nm)
- Worst Case: O(nm)

This means the running time grows quickly when searching for long text and patterns and when the algorithm compares the same characters many times.

22.2 KMP Algorithm Order of Growth

O(n+m)

For all cases, which means that it grows linearly with the input size and remains efficient and stable because it uses an LPS array to avoid repeated comparisons.

22.3 Linear Growth vs. Quadratic Growth

22.4 Linear Growth

Linear growth means the running time increases proportionally with the input size.

Example: O(n) like Naive best case and O(n+m) like KMP.

22.5 Quadratic Growth

Quadratic growth means the running time increases much faster as the input size grows.

Example: O(nm) like Naive Average and Worst Cases because it repeatedly compares characters.

22.6 Which Algorithm Grows Faster?

The Naive algorithm grows faster than KMP because it performs repeated comparisons, restarts matching after every mismatch, and does not store previous matching information.
KMP grows more slowly because it uses the LPS array, skips unnecessary comparisons, and never moves the text pointer backward.
So Naive becomes slower for large datasets. unlike KMP, which remains more efficient and scalable.

23. Final Comparison Between Naive and KMP Algorithms

Feature	**Naive Algorithm**	**KMP Algorithm**
Algorithm Type	Simple String Matching	Optimized String Matching
Main Technique	Direct Character Comparison	LPS Preprocessing
Best Case Complexity	O(n)	O(n + m)
Average Case Complexity	O(nm)	O(n + m)
Worst Case Complexity	O(nm)	O(n + m)
Space Complexity	O(1)	O(m)
Extra Memory Used	None	LPS Array
Growth Type	Quadratic Growth	Linear Growth
Repeated Comparisons	Yes	No
Preprocessing Required	No	Yes
Efficiency on Large Data	Lower	Higher
Main Advantage	Simple Implementation	Faster and Efficient Search
Main Limitation	Repeated Comparisons	Extra Preprocessing

Table 27: Final Comparison Between Naive and KMP Algorithms.

Experimental results show that the KMP algorithm outperforms the simple algorithm, especially as the input size increases. The primary reason is that the KMP algorithm does not perform repeated comparisons using an LPS array, whereas the simple algorithm compares the same characters multiple times.

24. Graph Analysis

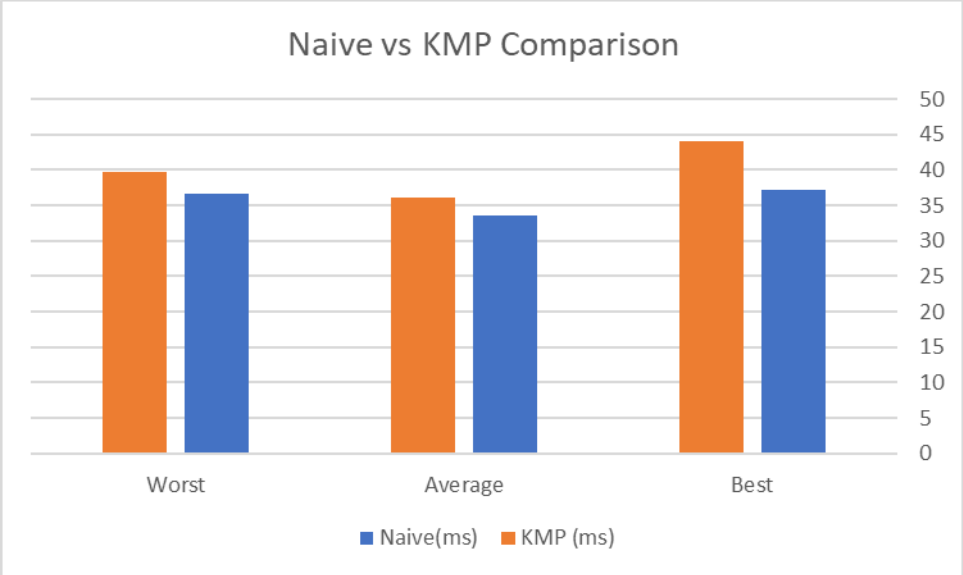

Figure 8: Comparison of Naive and KMP Running Time in Best, Average, and Worst Cases

The graph shows the time taken by Naive and KMP algorithms in best case, average case and worst case. The results reveal that generally KMP algorithm is faster and more efficient than the Naive algorithm. This improvement is granted due to the fact that KMP algorithm skips the un needed comparisons when searching with the help of LPS (Longest Prefix Suffix) array.

The Naive algorithm, on the other hand, does so many comparisons when it finds a mismatch; this slows down the execution time. It is easy to see how the difference between the two algorithms becomes apparent in the case of repeated partial matches occurring often.

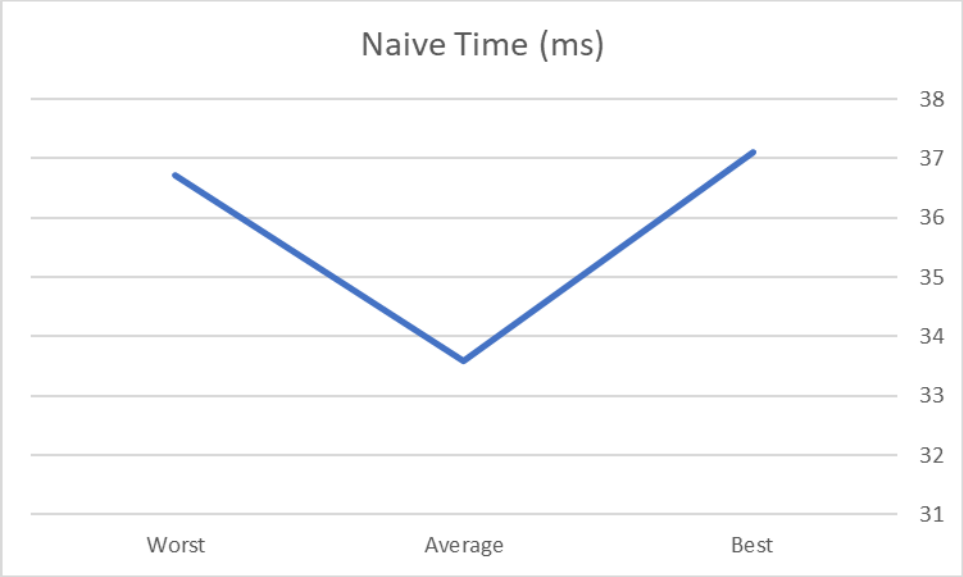

Figure 9: Naive Algorithm Running Time for Best, Average, and Worst Cases

This graph plots the Naive algorithm's performance for various input cases. The best case requires less execution time since the pattern is matched early or there are mismatches.

average case: the pattern occurs somewhere in the middle of the text; the execution time would become moderate since there would be more comparisons.

The least favorable case is the case where the algorithm has to make many comparisons between pattern and text positions before it terminates, and takes the longest time. Repeated comparisons have a significant impact on the performance of Naive algorithm because it restarts its comparison process from the beginning after each mismatch.

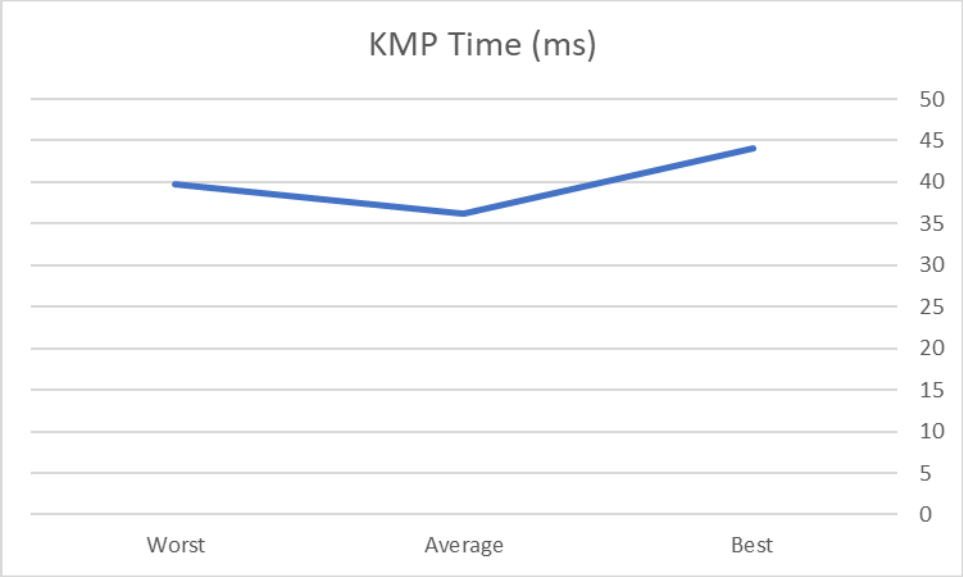

Figure 10: KMP Algorithm Running Time for Best, Average, and Worst Cases

As illustrated in Figure 11, the running times for the KMP algorithm for best, average, and worst cases are shown. Figure 11 shows the running time of the KMP algorithm in the best, average, and worst cases.

The graph shows that the KMP algorithm is more steady in terms of performance in best case, average case and worst case. The increase of execution time is only a little in the worst case, however, this is less than the Naive algorithm.

This is because of the use of the LPS array, which allows KMP to avoid repeating comparisons from start to finish. This will minimize unnecessary comparisons to make the searching process more efficient and make execution go faster.

The graph verifies that KMP is more appropriate to deal with large text datasets and repeated pattern searches.

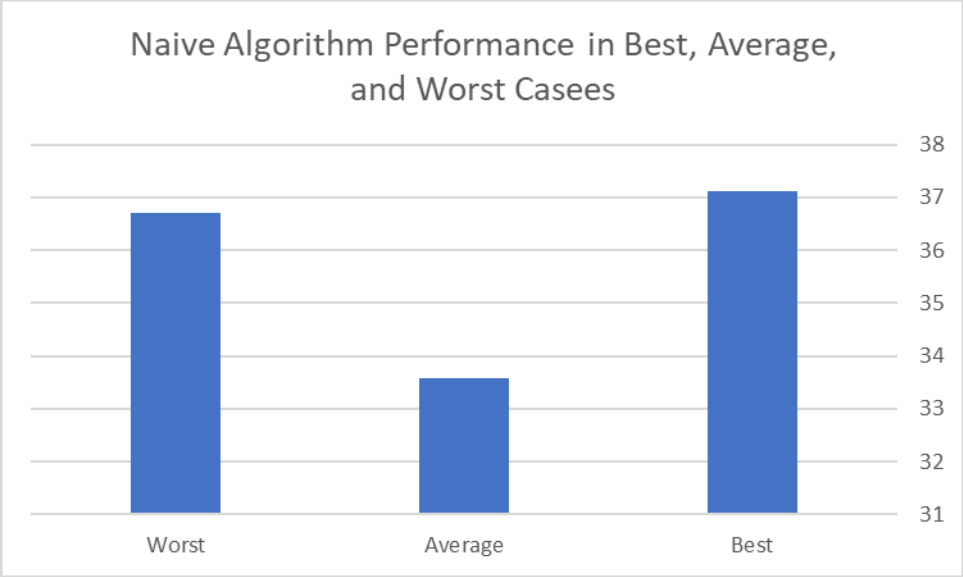

Figure 11: Performance Of The Naive Algorithm Execution Time For Best, Average, and Worst Cases.

This graph shows the working of the Naive algorithm under different searching conditions. In the worst case, the running time is significantly longer as the algorithm examines nearly all possible locations in the text.

The graph also illustrates that the algorithm is less efficient in the case of multiple partial matches. The more complicated the search, the more comparisons are made, and the slower the search will be.

For this reason the Naive algorithm is not as efficient for large-scale text searching applications.

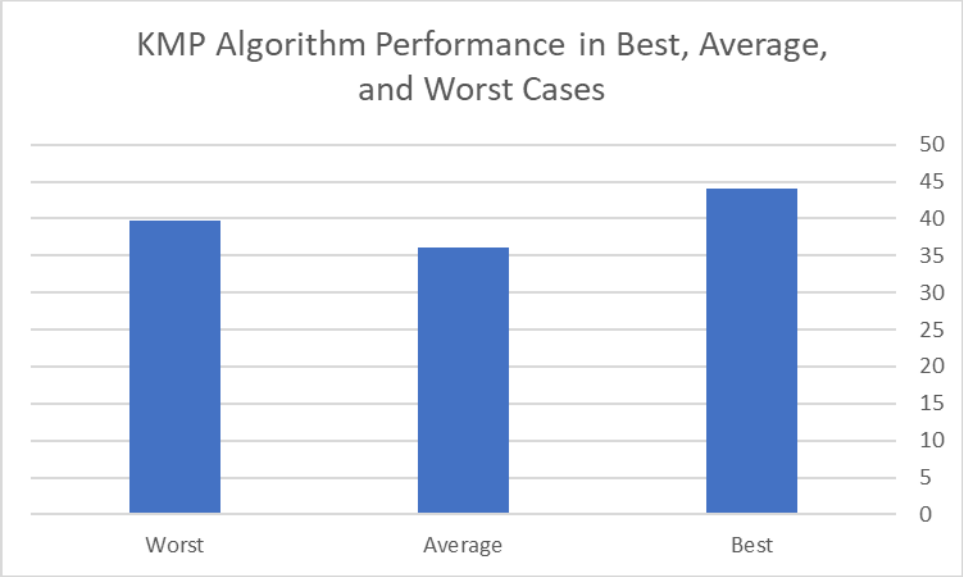

Figure 12: Performance Of The KMP Algorithm Execution Time For Best, Average, and Worst Cases.

Based on the graph, KMP algorithm performs better than Naive algorithm. The algorithm takes advantage of the pre-processing to optimize the searching process, and the execution time is relatively stable for different cases.

The number of comparisons is low and the KMP algorithm is efficient when the input becomes more complicated. This indicates that KMP is more appropriate for real-life text searching applications that require patterns to be repeated many times and large data sets.

25. T(n) Analysis

T(n) is used to describe the running time of an algorithm depending on the input size. It helps show how the performance of the algorithm changes when the text size becomes larger.

In this project, the Naive and KMP algorithms were tested using different input sizes such as small, medium, and large text documents.

For the Naive algorithm, the running time increases significantly as the input size grows because the algorithm compares the pattern with many possible positions inside the text. In some cases, repeated comparisons occur, which increases the total execution time.

On the other hand, the KMP algorithm performs more efficiently because it uses the LPS (Longest Prefix Suffix) array. This preprocessing step helps the algorithm skip unnecessary comparisons and reduces the searching time.

As a result, the KMP algorithm handles large text inputs better than the Naive algorithm. The difference between the two algorithms becomes clearer when using medium and large datasets.

The experimental results showed that the KMP algorithm required less execution time compared to the Naive algorithm, especially for larger inputs.

26. Conclusion

The aim of this project was to compare the Naive Text Searching algorithm and KMP Text Searching algorithm in the 20 Newsgroups Dataset. The two algorithms were programmed and evaluated with various examples of input, including the best, average and worst case scenarios.

The experimental results showed that the KMP algorithm outperformed Naive algorithm in terms of performance and time for medium and large size of text inputs. The main reason for this improvement is that KMP doesn't have to do redundant comparisons in the searching process.

The Naive algorithm, on the other hand, is simpler and easier to implement, but will perform worse with larger texts since it compares characters in a lot of different places within the text.

Finally, both of the developed algorithms were able to solve text searching problem, and the KMP algorithm was more efficient and appropriate in real-world text searching problems and large datasets. Hence, KMP is the improved option for large scale text classification systems to enhance the searching performance and to decrease the time of execution.

References

[1] D. E. Knuth, J. H. Morris Jr., and V. R. Pratt, “Fast Pattern Matching in Strings,” SIAM Journal on Computing, vol. 6, no. 2, pp. 323–350, 1977.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[3] T. Lecroq, “Knuth-Morris-Pratt Algorithm,” String Matching Algorithms. [Online]. Available: http://www-igm.univ-mlv.fr/~lecroq/string/node8.html. [Accessed: May 22, 2026].

[4] Python Software Foundation, “time — Time access and conversions,” Python Documentation. [Online]. Available: https://docs.python.org/3/library/time.html. [Accessed: May 22, 2026].

[5] “Naive Algorithm for Pattern Searching,” GeeksforGeeks. [Online]. Available: https://www.geeksforgeeks.org/dsa/naive-algorithm-for-pattern-searching/. [Accessed: May 22, 2026].

[6] “KMP Algorithm for Pattern Searching,” GeeksforGeeks. [Online]. Available: https://www.geeksforgeeks.org/kmp-algorithm-for-pattern-searching/. [Accessed: May 22, 2026].

[7] “Order of Growth,” GeeksforGeeks. [Online]. Available: https://www.geeksforgeeks.org/gate/order-o-f-growth/. [Accessed: May 22, 2026].

[8] “Big-O Cheat Sheet,” Big-O Cheat Sheet. [Online]. Available: https://www.bigocheatsheet.com/. [Accessed: May 22, 2026].

[9] “20 Newsgroups Dataset,” Kaggle. [Online]. Available: https://www.kaggle.com/datasets/crawford/20newsgroups. [Accessed: May 22, 2026].

[10] University of Hawai‘i, “ICS 311 Topic 23: String Matching,” 2020. [Online]. Available: https://www2.hawaii.edu/~nodari/teaching/s20/Notes/Topic-23.html. [Accessed: May 22, 2026].

[11] Johns Hopkins University, “Fast Pattern Matching in Strings PDF Copy.” [Online]. Available: https://www.cs.jhu.edu/~misha/ReadingSeminar/Papers/Knuth77.pdf. [Accessed: May 22, 2026].

[12] S. S. Skiena, The Algorithm Design Manual, 3rd ed. Cham, Switzerland: Springer, 2020.

[13] A. V. Aho, J. E. Hopcroft, and J. D. Ullman, Data Structures and Algorithms. Reading, MA, USA: Addison-Wesley, 1983.

[14] “Python IDLE Documentation,” Python Software Foundation. [Online]. Available: https://docs.python.org/3/library/idle.html. [Accessed: May 22, 2026].

[15] “Text Classification,” IBM Documentation. [Online]. Available: https://www.ibm.com/topics/text-classification. [Accessed: May 22, 2026].